

ST5003CMD

LINUX FUNDAMENTAL

Operating Systems, Security,
and Networks

BSc (Hons) Computer Science with AI Softwarica College of IT & E-commerce

STUDENT NAME		Ayusha Sapkota
STUDENT ID		250023
SUBJECT LEADER		Rupak Rajbanshi

Table of content

A Practical Approach to Understanding System Internals.....	3
(Data Types and OS Memory Management).....	3
1. Introduction.....	3
2. Data Type Size Exploration.....	4
Objective.....	4
Step 1: Creating the C program.....	4
Step 2 : Running the program.....	5
Step 3 : Generating compilation stages.....	5
Step 4: Comparing the results.....	6
3. Memory Segment Address Mapping.....	7
Objective.....	7
Step 1: Creating the memory segment program.....	7
Step 2: Compiling the program.....	8
Step 3: Running the program.....	8
Step 4: Viewing /proc/[PID]/maps.....	9
4. Identifying the Memory Segments.....	10
5. Calculating the Stack–Heap Address Difference.....	10
6. Observation.....	11
7. Conclusion.....	11

List of Figures

Figure 1: C program for observing data type sizes.....	5
Figure 2: C program for observing data type sizes.....	5
Figure 3: Output showing data type sizes.....	6
Figure 4: GCC compilation stages and generated files.....	6
Figure 5: Comparison of C Data Type Sizes on 64-bit and 32-bit Systems.....	7
Figure 6: C program for observing process memory segments.....	8
Figure 7: Compilation of the memory segment program.....	9
Figure 8: Running memory segment program and obtaining the process ID.....	9
Figure 9: Process memory mappings viewed through /proc.....	10
Figure 10: Mapping program addresses to process memory segments.....	11
Figure 11: Calculation of the address difference between stack and heap.....	11

A Practical Approach to Understanding System Internals

(Data Types and OS Memory Management)

1. Introduction

This laboratory exercise explores how data types are represented in memory and how the operating system manages memory for a running process. The practical work focuses on two main areas: observing the memory size of different C data types and examining the memory segments used by a process.

The first part uses the `sizeof()` operator to determine how many bytes different data types occupy on the system. The second part examines the addresses of variables stored in the text, data, BSS, heap, and stack areas. The practical also uses the Linux `/proc` filesystem to observe the memory mappings of a running process.

2. Lab 1: Data Type Size Exploration

Objective

The objective of this practical is to determine the memory size occupied by common C data types using the `sizeof()` operator and compare the results with the expected sizes on a 64-bit system.

Step 1: Creating the C program

A C program named `datatype_sizes.c` was created inside the `lab4` directory. The program uses `sizeof()` to display the size of different data types.

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# nano datatype_sizes.c
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# cat datatype_sizes.c
#include <stdio.h>

int main() {
    printf("char: %zu bytes\n", sizeof(char));
    printf("short: %zu bytes\n", sizeof(short));
    printf("int: %zu bytes\n", sizeof(int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("pointer: %zu bytes\n", sizeof(void *));

    return 0;
}
```

Figure 1: C program for observing data type sizes

The program was compiled using GCC with the following command:

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4# gcc datatype_sizes.c -o datatype_sizes
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4#
```

Figure 2: C program for observing data type sizes

Step 2 : Running the program

The compilation completed without errors, creating an executable file named `datatype_sizes`. The program displayed the number of bytes used by each data type on the current 64-bit Linux system.

The observed results were:

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# ./datatype_sizes
char: 1 bytes
short: 2 bytes
int: 4 bytes
long: 8 bytes
long long: 8 bytes
float: 4 bytes
double: 8 bytes
pointer: 8 bytes
```

Figure 3: Output showing data type sizes

Step 3 : Generating compilation stages

The source code was also processed through different GCC compilation stages. These commands produced the preprocessed file, assembly file, and object file.

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# gcc -E datatype_sizes.c -o datatype_sizes.i
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# gcc -S datatype_sizes.c -o datatype_sizes.s
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# gcc -c datatype_sizes.c -o datatype_sizes.o
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# ls
README.md      datatype_sizes.c  datatype_sizes.o
datatype_sizes  datatype_sizes.i  datatype_sizes.s
```

Figure 4: GCC compilation stages and generated files

Step 4: Comparing the results

The results were compared with the data type sizes expected on a 64-bit system. Most values matched the typical 64-bit representation. The `long` data type uses 8 bytes on the current 64-bit Linux system, whereas it can use 4 bytes on some 32-bit systems. This difference occurs because the size of `long` depends on the system's data model and architecture. Therefore, programs that rely on a specific size should use fixed-width integer types when portability is important.

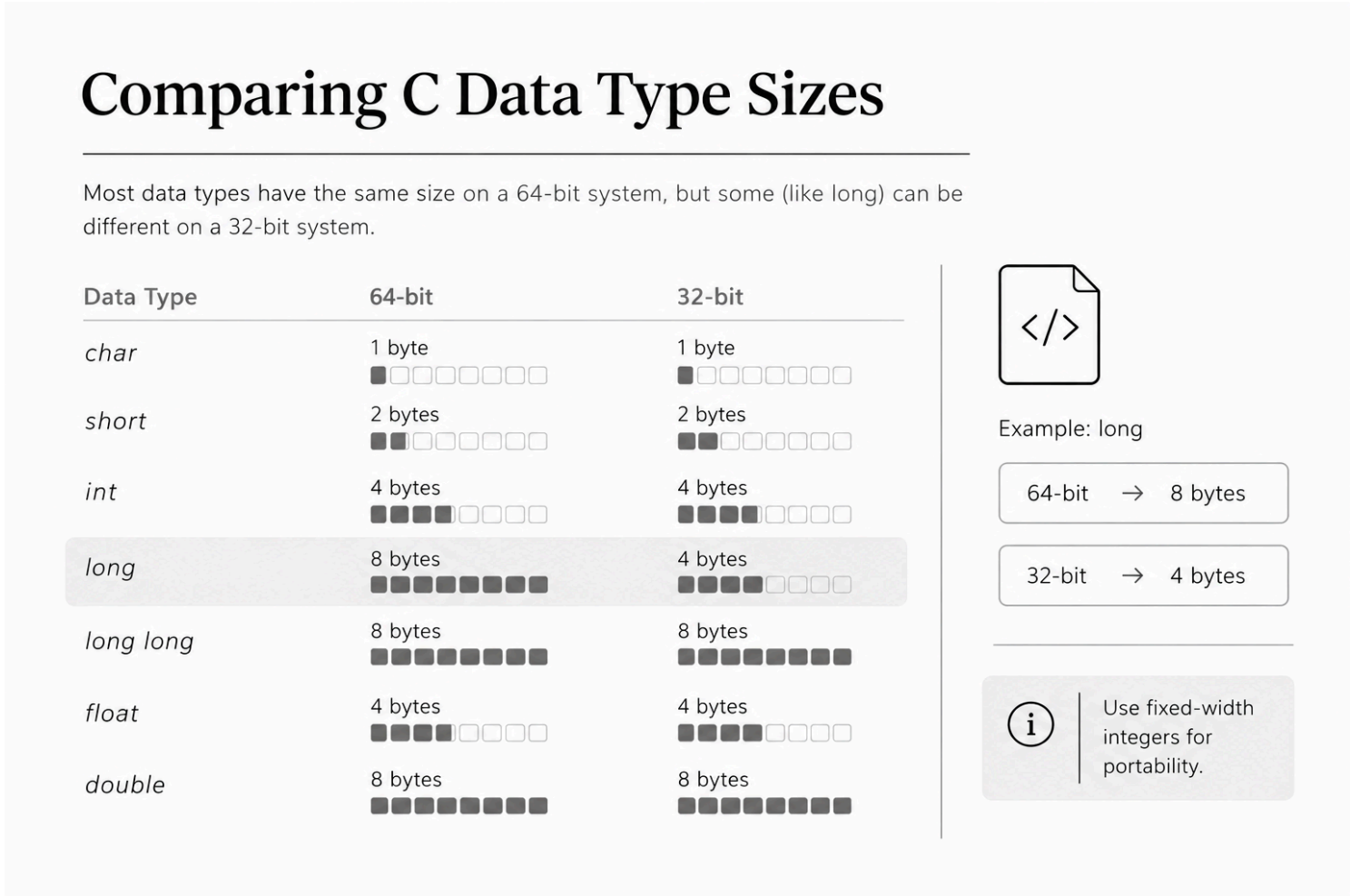


Figure 5: Comparison of C Data Type Sizes on 64-bit and 32-bit Systems

3. Lab 2: Memory Segment Address Mapping

Objective

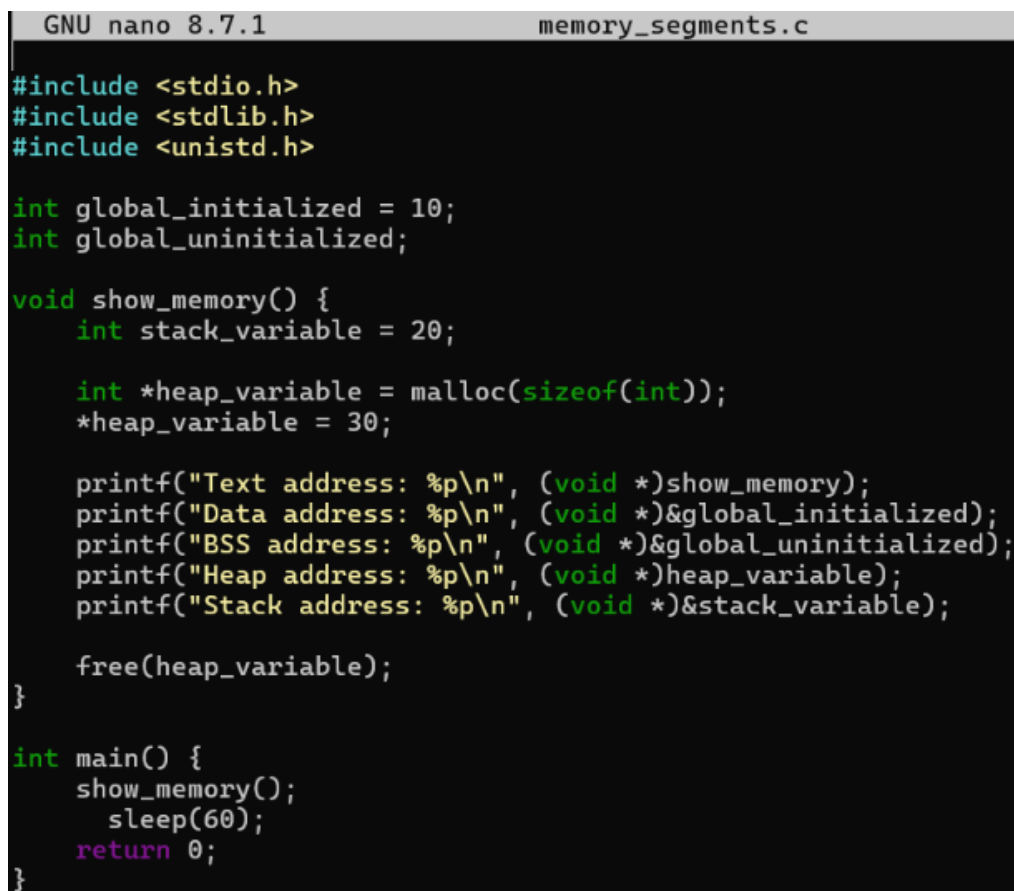
The objective of this practical is to observe how different variables and program instructions are located in the process address space and identify the memory segments used by the operating system.

Step 1: Creating the memory segment program

A C program named `memory_segments.c` was created. It contained:

- a. A global initialized variable for the **Data segment**
- b. A global uninitialized variable for the **BSS segment**
- c. A local variable for the **Stack**
- d. Dynamically allocated memory for the **Heap**
- e. A function address representing the **Text segment**

The program prints the memory address of each item.



```
GNU nano 8.7.1 memory_segments.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_initialized = 10;
int global_uninitialized;

void show_memory() {
    int stack_variable = 20;

    int *heap_variable = malloc(sizeof(int));
    *heap_variable = 30;

    printf("Text address: %p\n", (void *)show_memory);
    printf("Data address: %p\n", (void *)&global_initialized);
    printf("BSS address: %p\n", (void *)&global_uninitialized);
    printf("Heap address: %p\n", (void *)heap_variable);
    printf("Stack address: %p\n", (void *)&stack_variable);

    free(heap_variable);
}

int main() {
    show_memory();
    sleep(60);
    return 0;
}
```

Figure 6: C program for observing process memory segments

Step 2: Compiling the program

The program compiled successfully without errors.

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# nano memory_segments.c
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# gcc memory_segments.c -o memory_segments
```

Figure 7: Compilation of the memory segment program

Step 3: Running the program

The program was initially executed to observe the memory addresses. The program displayed addresses for the text, data, BSS, heap, and stack. To allow enough time to inspect the process, a `sleep(60)` statement was added to the program.

The program was then executed in the background, this produced a process ID (PID), which was required for inspecting the process through `/proc`.

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
# ./memory_segments &
[1] 695
Text address: 0x5f5ff11961a9
Data address: 0x5f5ff1199010
BSS address: 0x5f5ff1199018
Heap address: 0x5f6010139010
Stack address: 0x7ffc8cba19dc
[1]+  Done                  ./memory_segments
```

Figure 8: Running memory segment program and obtaining the process ID

Step 4: Viewing /proc/[PID]/maps

To observe the process memory map, the Linux /proc filesystem was used to inspect the memory mappings of the running process.

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4# cat /proc/963/maps
5f7b18744000-5f7b18745000 r--p 00000000 08:30 32597 /root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4/memory_segment
s
5f7b18745000-5f7b18746000 r-xp 00001000 08:30 32597 /root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4/memory_segment
s
5f7b18746000-5f7b18747000 r--p 00002000 08:30 32597 /root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4/memory_segment
s
5f7b18747000-5f7b18748000 r--p 00002000 08:30 32597 /root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4/memory_segment
s
5f7b18748000-5f7b18749000 rw-p 00003000 08:30 32597 /root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4/memory_segment
s
5f7b18749000-5f7b1874a000 rw-p 00000000 00:00 0 [heap]
73384ac0000-73384ac20000 r--p 00000000 08:30 13604 /usr/lib/x86_64-linux-gnu/libc.so.6
73384ac20000-73384ac40000 r-xp 00020000 08:30 13604 /usr/lib/x86_64-linux-gnu/libc.so.6
73384ac40000-73384ac60000 r--p 001c0000 08:30 13604 /usr/lib/x86_64-linux-gnu/libc.so.6
73384ac60000-73384ae12000 r--p 0020e000 08:30 13604 /usr/lib/x86_64-linux-gnu/libc.so.6
73384ae12000-73384ae14000 rw-p 00212000 08:30 13604 /usr/lib/x86_64-linux-gnu/libc.so.6
73384ae14000-73384ae21000 rw-p 00000000 00:00 0
73384af0000-73384af09000 rw-p 00000000 00:00 0
73384af09000-73384af11000 rw-p 00000000 00:00 0
73384af11000-73384af15000 r--p 00000000 00:00 0 [vvar]
73384af15000-73384af17000 r--p 00000000 00:00 0 [vvar_vclock]
73384af17000-73384af19000 r-xp 00000000 00:00 0 [vdso]
73384af19000-73384af1a000 r--p 00000000 08:30 13601 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73384af1a000-73384af49000 r-xp 00001000 08:30 13601 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73384af49000-73384af54000 r--p 00030000 08:30 13601 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73384af54000-73384af56000 r--p 0003b000 08:30 13601 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73384af56000-73384af57000 rw-p 0003d000 08:30 13601 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73384af57000-73384af58000 rw-p 00000000 00:00 0
7ffc04348000-7ffc04369000 rw-p 00000000 00:00 0 [stack]
```

Figure 9: Process memory mappings viewed through /proc

Here, 963 was the PID obtained when the program was running. The output showed different memory regions, including: Program executable regions, Heap, Stack.

The executable region with r-xp permissions represents the area containing executable program instructions. The (heap) and (stack) entries identify the heap and stack memory regions.

4. Identifying the Memory Segments

The addresses printed by the program were compared with the memory mappings obtained from `/proc/963/maps`.

The following observations were made:

Item	Address (example)	Memory Segment
Text/function	0x5f7b18745	Text
Global initialized variable	0x5f7b18748	Data
Global uninitialized variable	0x5f7b18748	BSS
Dynamically allocated variable	0x5f7b457f5	Heap
Local variable	0x7ffc04367	Stack

Figure 10: Mapping program addresses to process memory segments

The function address was located inside the executable `r-xp` region, confirming the Text segment. The global variables were located in the writable program area associated with Data/BSS. The dynamically allocated memory was located in the `[ heap ]` region, while the local variable was located in the `[ stack ]` region.

5. Calculating the Stack–Heap Address Difference

The stack and heap addresses were used to calculate their address difference. The calculation was performed using:

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/Lab4# python3 -c "print(hex(0x7ffc043672dc - 0x5f7b457f5010))"
0x2080beb722cc
[1]+  Done                  ./memory_segments
```

Figure 11: Calculation of the address difference between stack and heap

The result was: `0x2080beb722cc`

This shows the hexadecimal address difference between the observed stack and heap addresses.

6. Observation

The practical demonstration demonstrated that different parts of a process use different memory regions. The stack contained the local variable and had a relatively high address, while dynamically allocated memory was located in the heap at a lower address. The global variables were located in the Data/BSS area, while the program instructions were located in the Text segment.

7. Conclusion

This laboratory exercise provided a practical understanding of how data types and memory are handled by a computer system. The `sizeof ( )` experiment showed that data type sizes depend partly on the system architecture, while the memory mapping experiment demonstrated the separation of Text, Data, BSS, Heap, and Stack segments. The practical also showed how Linux provides information about a running process through the `/proc` filesystem.

1. **What is a data type?**

A data type defines what kind of value a variable can store, such as int, float, char, or double. It also determines how much memory is needed.

2. **Five main memory segments:**

The five main segments are Text, Data, BSS, Heap, and Stack.

3. **Difference between Data and BSS:**

The Data segment stores initialized global and static variables. The BSS segment stores uninitialized or zero-initialized global and static variables.

4. **Why is the stack at high addresses and text at low addresses?**

This arrangement allows the stack to grow downward and the heap to grow upward, giving them space to grow without immediately overlapping.

5. **What happens to stack memory when a function returns?**

The function's stack frame is automatically removed when it returns. The stack pointer is adjusted, effectively reclaiming that memory.

6. **How does the OS know which memory belongs to which process?**

The OS uses virtual memory and page tables to map each process's virtual addresses to physical memory. Each process has its own separate address space.